# FinPilot Architecture

**Status:** Locked
**Last Updated:** March 31, 2026
**Full Design Details:** See DESIGN.md

---

## What Is FinPilot?

A full-stack personal finance app — two separate repos talking over REST:

- **finpilot-backend** — Node.js + Express + TypeScript + Prisma + PostgreSQL
- **finpilot-frontend** — React + TypeScript + Vite + Tailwind CSS

Types are shared via an npm package ( `@finpilot/api-types` ) published by the backend.

---

## Backend Layers

```
Request → Routes → Services → Repositories → PostgreSQL
```

| Layer | Location | Does | Does NOT |
|---|---|---|---|
| **Routes** | `src/api/routes/` | Parse HTTP, validate with Zod, return JSON | Contain business logic |
| **Services** | `src/services/` | Business logic, orchestrate repos, workflows | Know about HTTP or query DB directly |
| **Repositories** | `src/repositories/` | Database queries via Prisma | Make business decisions |
| **Models** | `prisma/schema.prisma` | Define DB structure, generate type-safe client | — |

**Why?** Each layer has one job. Services are testable without a database (mock the repo). Routes are thin. Repositories are reusable.

---

## Frontend Layers

```
Pages → Components → Hooks → TanStack Query / Zustand → Axios → Backend API
```

| Layer | Location | Does |
|---|---|---|
| **Pages** | `src/pages/` | One per route, compose components |
| **Components** | `src/components/` | Reusable UI (forms, tables, charts, cards) |
| **Hooks** | `src/hooks/` | Data-fetching logic (wraps TanStack Query) |
| **Store** | `src/store/` | UI-only state via Zustand (modals, filters) |
| **API Client** | `src/services/api.ts` | Axios with JWT interceptor |

**Rule:** Components never call APIs directly — they use hooks. TanStack Query handles server state (caching, refetch). Zustand handles UI state only.

---

## Five Key Decisions

### 1. Separate Repos (not monorepo)

Independent deployments, team autonomy, clear API contract.
**Trade-off:** Types synced via npm package instead of auto-shared.

### 2. Base Currency at Write Time

Store `baseCurrencyAmount` + `exchangeRate` when a transaction is created.
**Why:** Aggregations are instant, historical accuracy preserved, audit-compliant.
**Trade-off:** Extra column per transaction.

#### Write-Time vs Read-Time Conversion

**Write-time** (what we do): convert the amount using today's exchange rate when the transaction is saved. Store both the original amount/currency and the converted base-currency amount alongside the rate that was used.

**Read-time** (the alternative): store only the original amount/currency and convert on every query using the latest rate.

| Concern | Write-Time | Read-Time |
|---|---|---|
| Query performance | Fast — aggregates use pre-converted column | Slow — every row needs a conversion lookup |
| Historical accuracy | Locked to the rate when the event occurred | Retroactively changes past totals |
| Audit / compliance | Rate is immutable evidence | No record of what rate applied |
| Storage cost | Two extra columns per transaction | No extra columns |
| Currency change | Past transactions keep old base currency | All transactions reflect new base instantly |

**Why write-time wins for FinPilot:** Financial apps need deterministic totals. If a user views their March report in March and again in June, the numbers must match. Read-time conversion silently rewrites history whenever rates fluctuate, which breaks trust and audit trails. The storage overhead (two extra columns) is negligible compared to the correctness guarantee.

### 3. AI Gets Aggregated Data Only

Pre-aggregate monthly summaries before sending to AI. Never send raw transactions.
**Why:** Privacy, compliance, lower cost.

### 4. Dashboard Computed On-Demand

No caching layer — query totals directly from PostgreSQL on each request.
**Why:** Simple for MVP, always fresh. Add Redis/materialized views later if needed.

### 5. JWT in HttpOnly Cookies

Stateless auth, no server-side sessions.
**Why:** Scales horizontally, XSS-resistant, CORS-friendly for separate repos.

---

## Entities (7)

| Entity | Purpose |
|---|---|
| **User** | Account, email, password hash, base currency preference |
| **Category** | Hierarchical (parent/child), typed as INCOME or EXPENSE |
| **Transaction** | Amount in original + base currency, linked to category |
| **Budget** | Per-category per-month spending limit |
| **ExchangeRate** | Historical rates for audit trail |
| **BankSyncLog** | Mock bank sync history |
| **AuditLog** | Who changed what, when (compliance) |

---

## Elevator Pitch

> *"FinPilot has two repos: a backend API and a React frontend that communicate over REST. The backend uses three layers — routes for HTTP handling, services for business logic, and repositories for database access. This makes it easy to test and maintain. For financial accuracy, we convert currency at write time so aggregations are instant and historically correct. AI only sees monthly summaries, never raw data, for privacy. Dashboard is computed on-demand to keep things simple. Auth uses stateless JWT in httpOnly cookies."*